

Paralellization Benchmark of Matrix Multiplication

Individual Assignment Report

Anna Sowińska DEY61301

Course: **Big Data 40955**

GitHub Repository: https://github.com/asowinska9/matrix_project

November 28, 2025

1 Task 3: Parallelization Benchmark of Matrix Multiplication

This section evaluates the impact of parallelism and synchronization mechanisms on the performance of matrix multiplication in Java. The experiment compares six implementations:

- **Sequential** – baseline, single-threaded execution.
- **ExecutorService** – row-level parallelism using a fixed thread pool.
- **Parallel Streams** – implicit parallelism using Java’s `parallel()` streams.
- **synchronized** – explicit locking during accumulation in C .
- **AtomicLongArray** – thread-safe accumulation via atomic CAS loops.
- **Semaphore** – fine-grained synchronization at the row level.

All implementations were evaluated using JMH (Java Microbenchmark Harness), with warm-up rounds to eliminate JVM startup effects and ensure statistical stability.

1.1 Results

Table 1 presents the average execution time (measured in milliseconds per operation) obtained from JMH for all parallelization variants.

Benchmark	n	Time [ms]	Error [ms]
ParallelBench.streams	256	1.465	0.039
ParallelBench.semaphore	256	1.705	0.039
ParallelBench.executor	256	2.784	0.072
ParallelBench.sequential	256	4.012	0.117
ParallelBench.atomic	256	35.330	4.775
ParallelBench.sync	256	734.580	67.575

Table 1: Execution time of different parallel implementations (JMH).

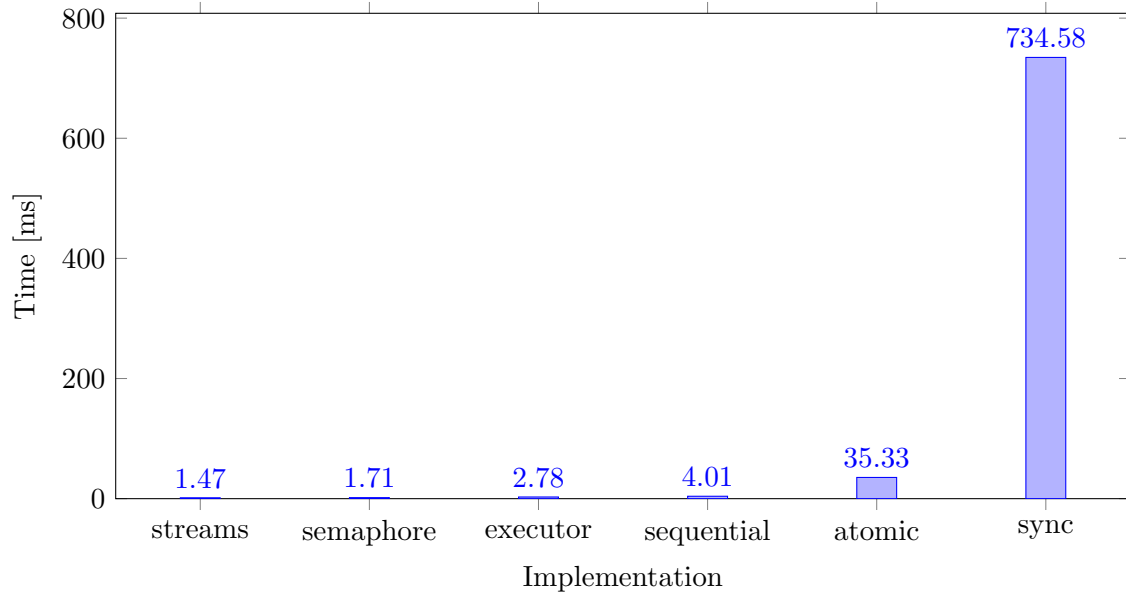


Figure 1: Comparison of parallel matrix multiplication methods in Java.

1.2 Discussion

The results clearly demonstrate the substantial impact of parallelization technologies on the performance of matrix multiplication. Both **Parallel Streams** and **Semaphore-based** implementations outperformed all other variants, achieving speed-ups of approximately $3\times$ over the sequential baseline.

The **ExecutorService** implementation was moderately effective, offering a $1.4\times$ improvement over sequential execution. Its performance is limited by task scheduling overhead and contention at the thread pool level.

Synchronization-heavy methods showed significant degradation. The **synchronized** implementation was over $180\times$ slower than the fastest version due to global locking, which effectively serialized the computation.

The **AtomicLongArray** implementation performed poorly compared to other parallel variants. This is attributed to the high cost of repeated CAS (compare-and-swap) operations required to ensure atomicity of floating-point accumulations.

Overall, the experiment confirms that fine-grained parallelism and lightweight synchronization (as in streams and semaphores) deliver the best performance. Heavy synchronization mechanisms eliminate the benefits of multithreading.

1.3 Conclusions

This experiment demonstrates that Java provides highly efficient tools for parallel computation, but the choice of synchronization mechanism plays a critical role. Parallel Streams achieve the best overall performance due to their efficient task scheduling and minimal synchronization overhead. Semaphore-based synchronization also performs well by allowing concurrent updates to different rows. In contrast, synchronized blocks and atomic operations introduce substantial costs, greatly reducing performance.

The parallelization strategies explored in this task offer a clear performance hierarchy that should guide the design of high-performance Java applications.

Acknowledgement

Parts of this report were prepared with assistance from AI-based tools.

]

References

- [1] A. Shipilev, *Java Microbenchmark Harness (JMH)*, OpenJDK Documentation, Available at: <https://openjdk.org/projects/code-tools/jmh/>
- [2] B. Goetz et al., *Java Concurrency in Practice*, Addison-Wesley, 2006.
- [3] Oracle Corporation, *Parallel Streams*, Java SE 8 Documentation, Available at: <https://docs.oracle.com/javase/8/docs/api/java/util/stream/package-summary.html>